MEMORY MANAGEMENT

October 19, 2023

UNIVERSITY OF MARYLAND
A. JAMES CLARK SCHOOL OF ENGINEERING

Agenda

- Memory Management Basics
- Secure Memory Management Rules
- Mitigations and Countermeasures

Memory Management Basics

Background and Concepts

- Raw memory
  - A large array of words or bytes
  - Each word/byte has its own address.
  - Program counter stores the address of the instruction to be executed.
  - Executing an instruction may cause operands to be fetched from memory.
  - The memory unit sees a stream of memory addresses.
    - It does not know how or why the addresses are generated.

Address Binding

- Assume that a process can reside in any part of the physical memory.
- How do we specify the memory addresses that the program uses?
- When the compiler generates executable, it must specify the addresses.
  - e.g ‘load R1, mem[100]’
- Addresses used in the executables are logical addresses.
- Logical addresses may or may not be physical addresses, that addresses for raw memory

Address Binding

- Mapping from logical address (used in user programs) to physical addresses (on raw memory)
  - This can affect how addresses in an executable are generated (or how the executables are generated by the compiler).
- What kind of address does the CPU see, logical or physical?

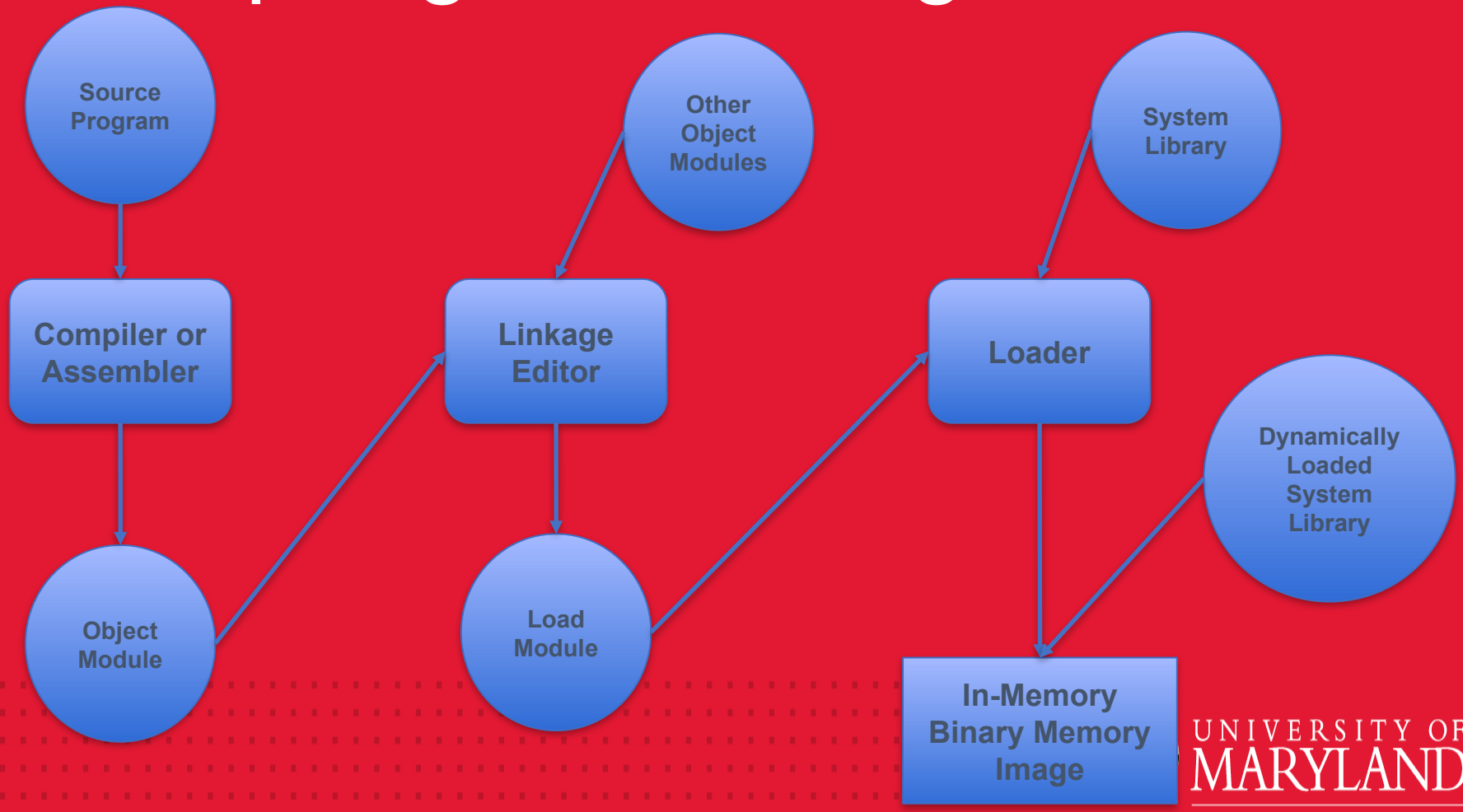
When is Address Binding Done?

- Steps to get our source program to run?
  - Compile ☾ .o files
  - Link ☾ a.out
  - Load (after you type ‘a.out’).
- Depending on how memory is managed, address binding can be done in any of the steps:
  - Compile time (compile/link). When the compiler knows the address where the program will be loaded, it can produce the absolute code.
    - MS-DOS, or some embedded systems.
    - Logical address = physical address
  - Load time. When the process can be placed anywhere in the memory. The compiler can only generate relocatable code.
    - Physical address = base address + logical address
  - Execution time. The process can be moved during execution.
  - Binding can only be done at run time. Need hardware support.

When is Address Binding Done?

- Compile time, load time, and execution time, which one is the most common one on the current machines?
  - Execution time through hardware support for virtual memory.
  - In a typical virtual memory system:
    - Logical (virtual) memory for a process is a continuous block (e.g. 0 – 2^32 bytes)
    - Physically, different pages can be mapping to different locations of the physical memory.

Compiling to Running



Dynamic Loading

- So far, we have assumed that the whole program and data for the program must be in memory in order for a process to execute.
- Problem?
  - Even if your code is small, the memory needed will be large.
  - Why?

» Each program must be linked with the whole standard library

» The whole library must be in memory even though most functions in that library are not used.

Dynamic Loading

- Don’t load the whole program into memory before we run a program
- Just load the main routine at the beginning. When a new routine is called, it is loaded into the memory.
- The executable will still be large, but the memory used will be smaller.
  - Routines that are not used will not be in the memory.
  - Dynamic loading can introduce significant time penalty.
  - Space/time trade-off

Libraries

- With dynamic loading, the actual memory used may not be large, but the executable size is large. A program must be statically linked with all libraries.
- Dynamic linking is conceptually similar to dynamic loading (except apply to linking
  - Postpone the linking until execution.
    - How it works:
      - Use a stub to replace the whole library routine
      - Enough information to locate the library routine.
      - When the stub is executed, it replaces itself with the address of the routine and executes the routine.
    - Only one copy of library code is needed for all programs.
    - Such a library is also known as a shared library.
- The executable is much smaller with such libraries.
- Windows DLL files (.dll) and UNIX shared object (.so) libraries

Dynamic Linking and Shared Libaries

- Try ‘gcc a.c’ and ‘gcc –static a.c’ and check the size of the executable.
- One problem with dynamically linked executable is that it is less portable: assume the shared library is available and the same
- Statically linked executables are more self-contained.

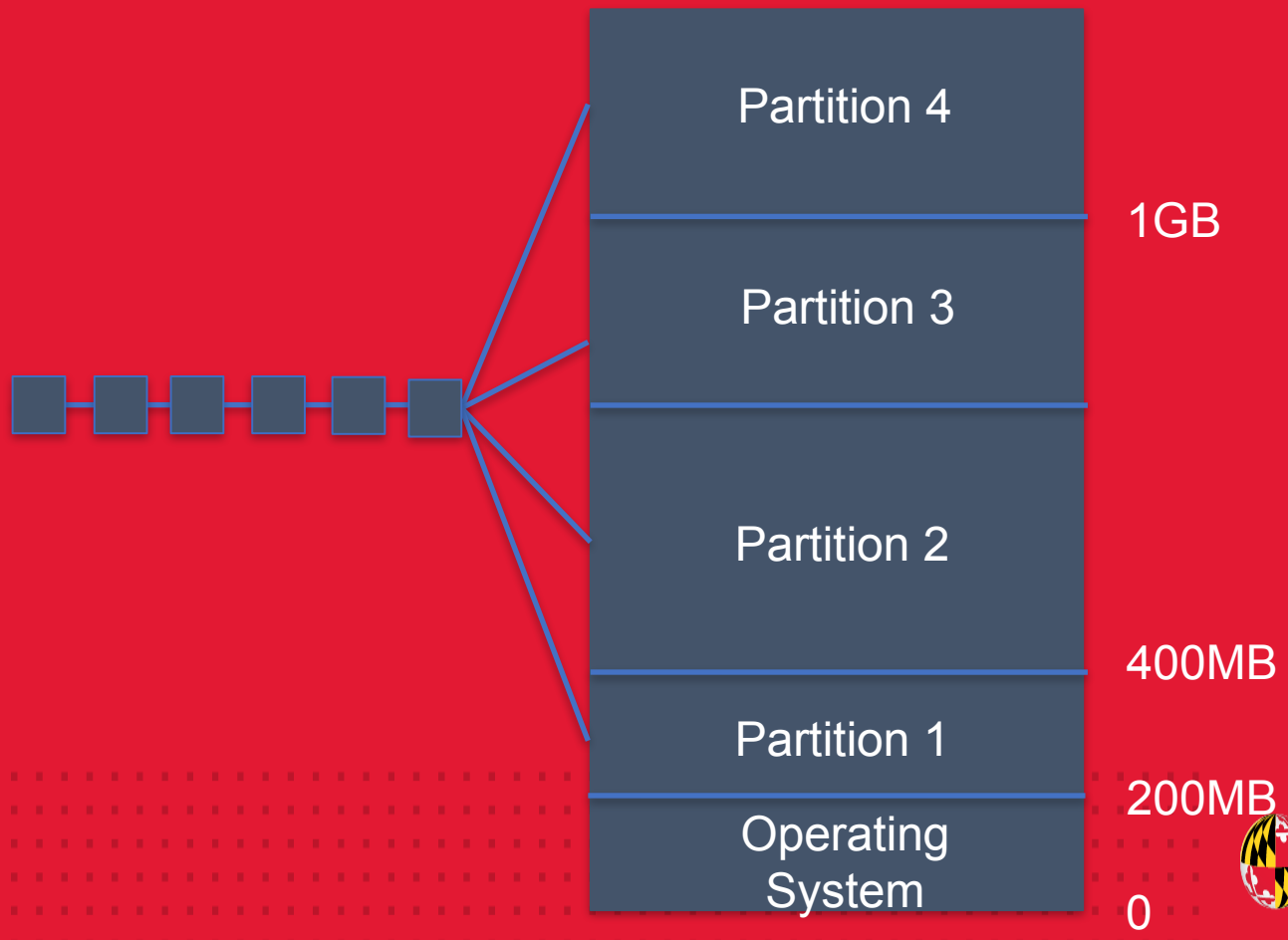
Swapping

- The OS may choose to swap a process out of memory temporarily to swap space.
- Swap space is on disk (see ‘top’)

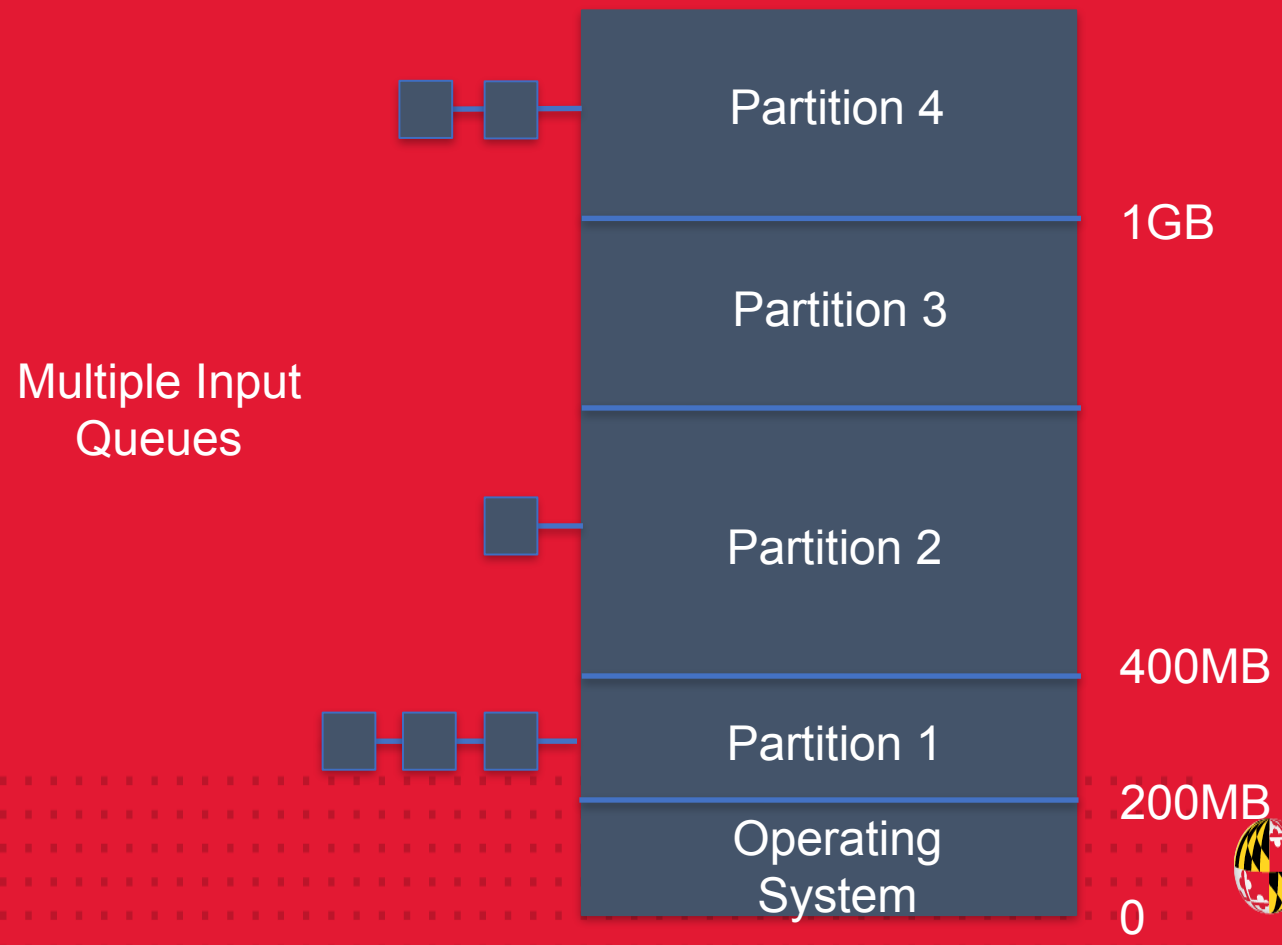
Fixed partitions

- Divide memory up into n partitions
- Can be done when starting up
- New job arrives
  - Place into input queue
  - Wait for the smallest partition that is large enough
  - Drawback – unused space in a partition is lost

Fixed Partitions



Fixed partitions

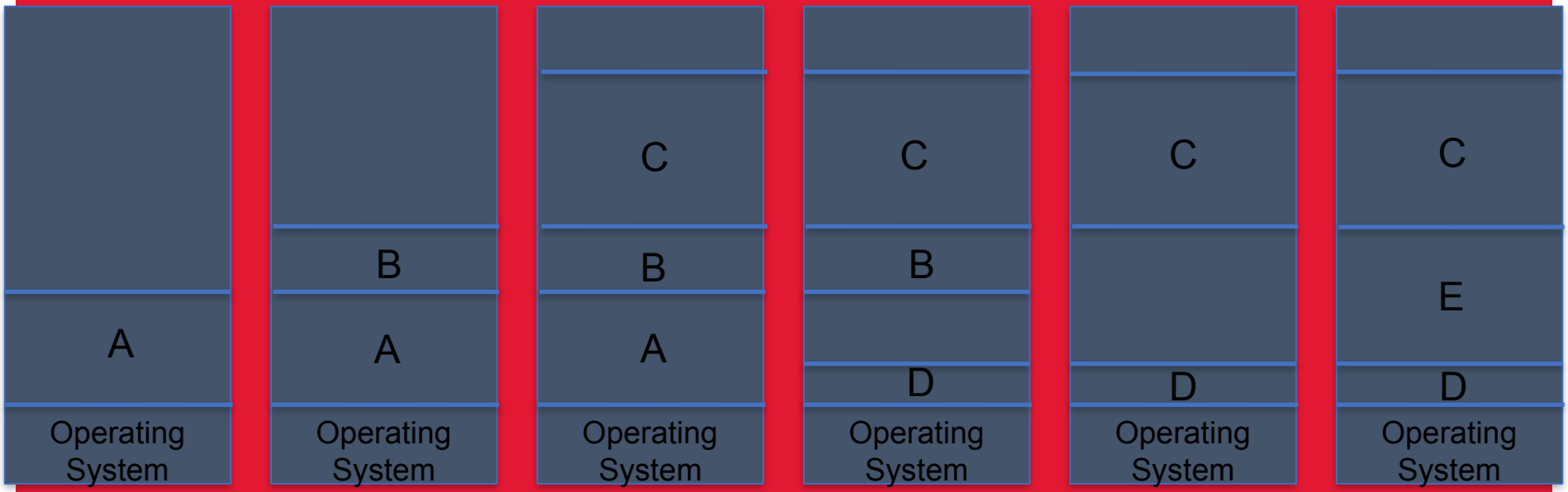


UNIVERSITY OF MARYLAND
A. JAMES CLARK SCHOOL OF ENGINEERING

Problems

- Two problems
  - Relocation
  - Protection
- Linker must know where the program will begin in memory
- One solution is Program Status Word
  - 4 bit protection code to each block of memory
  - Trap any attempt by program to access memory whose PSW didn't match
- Alternate Solution
  - Base – base register loaded with the start of the partition
  - Limit – length of the partition

Dynamic Partitioning



Keeping track of things

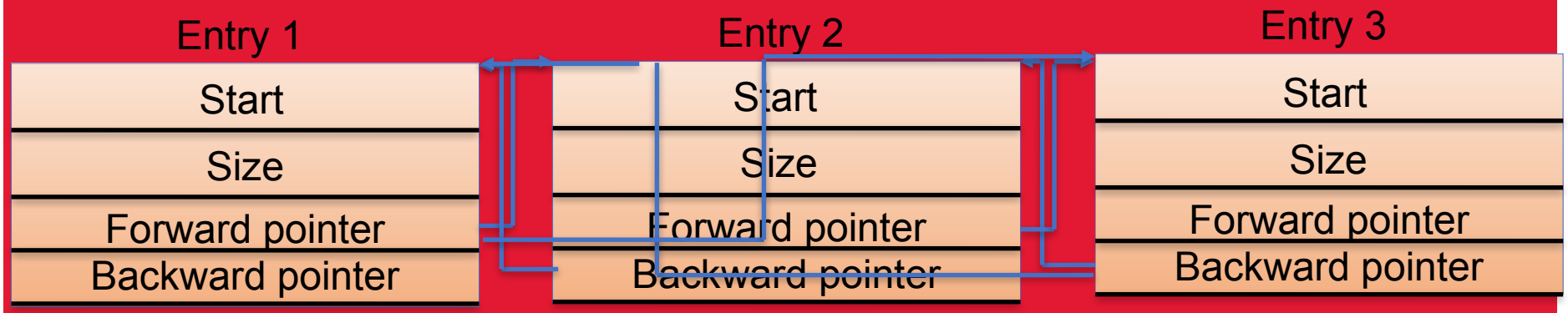
- Two ways to keep track of things
  - Bit maps
  - Free Lists

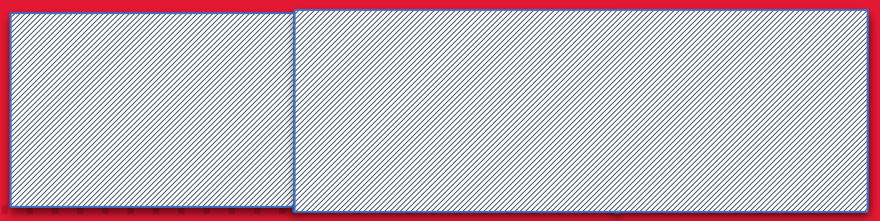
Bit maps

1	1	1	1	1	0	0	0
1	1	1	1	1	1	1	1
1	1	1	1	1	0	0	0

Free List

Linked lists





Algorithms Used to Allocate Memory

- First Fit
  - Find the first hole big enough
- Next Fit
  - Similar to First Fit – keeps track of it's location in the list
  - Slightly worse performance than First Fit
- Best Fit
  - Searches the entire list and takes the smallest hole that is adequate
- Worst Fit
  - Like Best Fit but takes the largest hole
- Quick Fit
  - Maintains separate lists for the most common size of holes requested
- All of the top four can be speeded up by keeping separate lists of the processes and holes
  - Tradeoff - complexity

Virtual Memory

- Virtual Memory
  - Combined size of the program, data and stack may exceed the amount of physical memory available
  - OS keeps things that are in use in memory with other portions on disk
- Utilize an MMU
  - Maps virtual addresses to physical addresses

Virtual Memory

- Page Tables
  - Virtual address consists of virtual page number and offset
  - Virtual page number is an index into page table
  - Page table entry provides the page frame number
  - Added to high-order end of the offset to form a physical address
- Two issues
  - Page table may be very large
  - Mapping must be fast

Other Page management

- Multi-level page tables
- Translation Lookaside buffers
  - Stores recent translations
  - Fully within the MMU

Protecting Memory and Addressing

- Isolate processes from one another and from the OS.
  - Fence
  - Relocation
  - Base/Bounds Registers
  - Tagged Architecture
  - Segmentation
  - Paging

Fence

- A fence is a method to confine users to one side of a boundary.
- Usually, fence is implemented via a hardware register.

Relocation

- Relocation is the process of taking a program written as if it began at address 0 and changing all addresses to reflect the actual address at which the program is located in memory.

- Fence register can be used within relocation process. To each program address, the contents of the fence register are added. This both relocates the address and guarantees that no one can access a location lower than a fence address.

Base/Bounds Registers

- In a multiuser, multiprogramming environment, fence register is variable. In this case fence register is called base register.
- Fence registers only provide a lower bound (a starting address), but not an upper one.
  - A second register, called a bounds register can be used to provide a upper bound.
  - In this way, a program's addresses are neatly confined to the space between the base and the bounds registers.
- This technique protects a program's addresses from modification by another user.

Tagged Architecture

- The disadvantage of Base/Bounds technique
- Tagged Architecture
  - Every word of machine memory has one or more extra bits to identify the access rights to that word.
  - This technique is not wide spread because of the market consideration (compatible).

Segmentation

- Segmentation divides a program into separate pieces. Each piece has a logical unity, a relationship among all of its code or data value.

- Segmentation was developed as a feasible means to have the effect of an unbounded number of base/bounds registers: a program could be divided into many pieces having different access rights.

- The operating system must maintain a table of segment names and their true addresses in memory. The program address is in the form <*name, offset*>. OS can retrieve the real address via looking for the table then making a simple calculation:

address of the name + offset

Paging

- An alternative to segmentation is paging.
  - Program is divided into equal-sized pieces called pages,
  - Memory is divided into the same sized units, called page frames.
    - Each address is represented in a form <*page, offset*>.

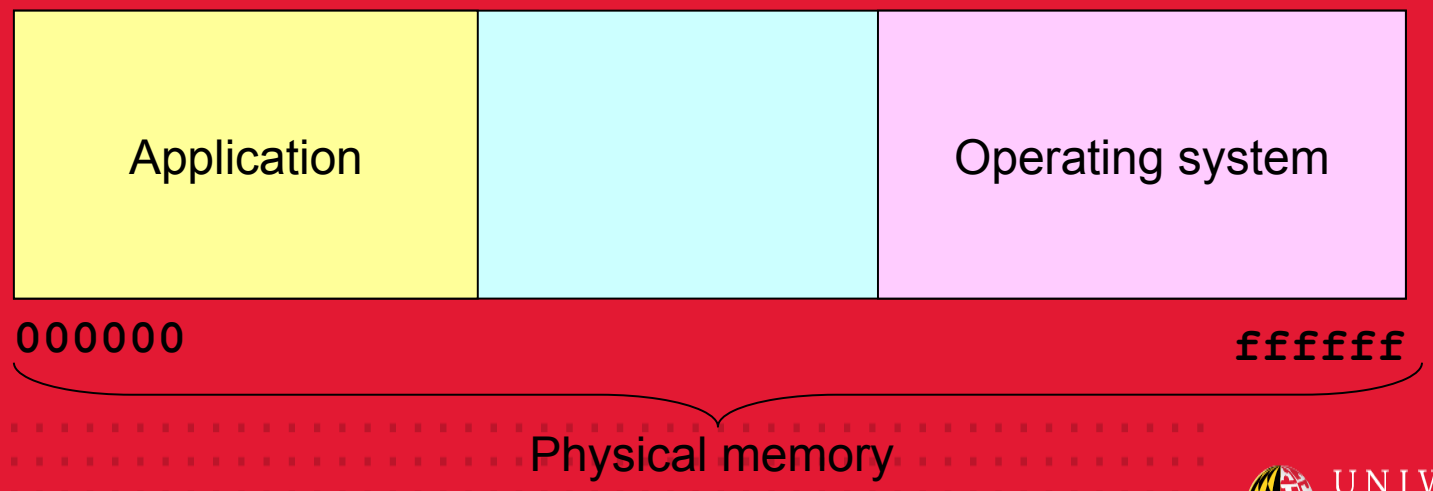
- Operating system maintains a table of user page numbers and their true addresses in memory.
  - The *page* portion of every <*page, offset*> reference is converted to a page frame address by a table lookup
  - The *offset* portion is added to the page frame address to produce the real memory address of the object referred to as <*page, offset*>.

Uniprogramming Without Memory Protection

- Simplest.
- Each application runs within a hardwired range of physical memory addresses
- One application runs at a time
- Application can use the same physical addresses every time, across reboots

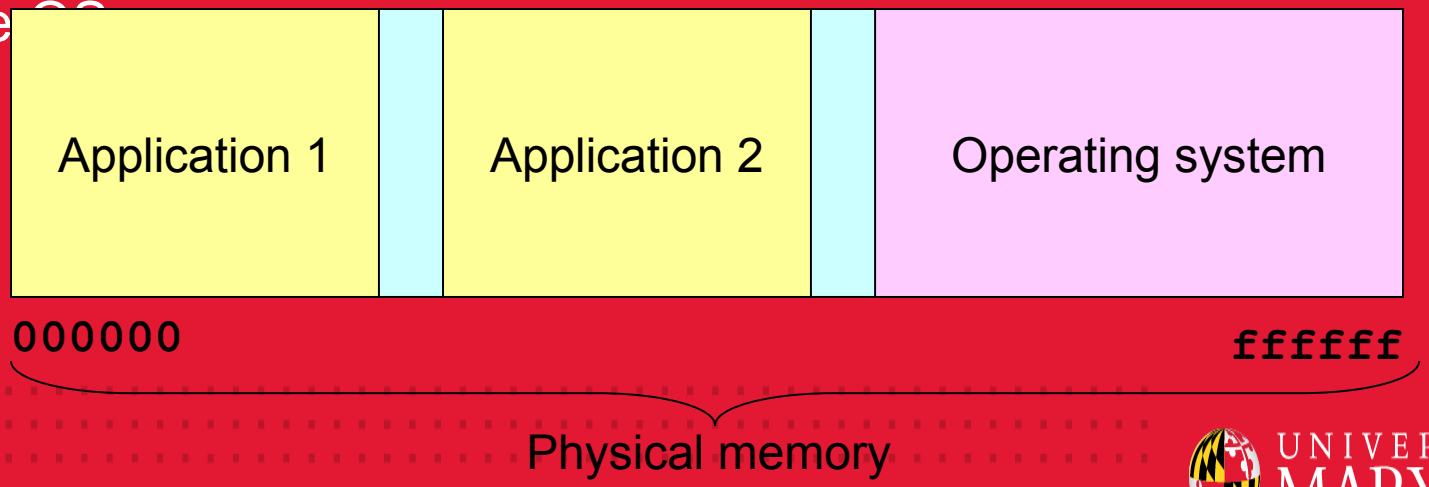
Uniprogramming Without Memory Protection

- Applications typically use the lower memory addresses
- An OS uses the higher memory addresses
- An application can address any physical memory location



Multiprogramming Without Memory Protection

- When a program is copied into memory, a ***linker-loader*** alters the code of the program (e.g., loads, stores, and jumps)
- To use the address of where the program lands in memory
- Bugs in any program can cause other programs to crash, even the OS



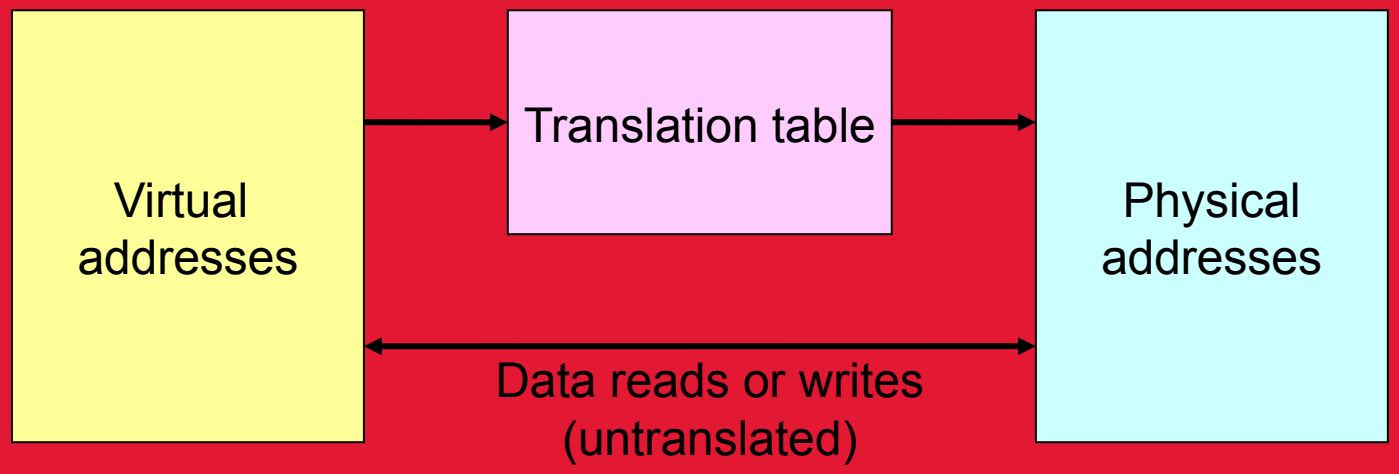
Multiprogrammed OS With Memory Protection

- ***Memory protection*** keeps user programs from crashing one another and the OS
- The big idea: make all memory accesses go through an OS controlled agency
  - The agency provides services (address translation) while checking all addresses
- Two hardware-supported mechanisms
  - Address translation (the OS controlled agency)
  - Dual-mode operation

Address Translation

- Each process is associated with an **address space**, or all the addresses a process can touch
  - Each process believes that it owns the entire memory, starting with the *virtual* address 0
- The missing piece is a translation table to translate every memory reference from virtual to physical addresses

Address Translation Visualized



More on Address Translations

- Address translation example:
  - Translation table maintains <base, bound>
  - If (virtual address > bound) error;
  - Else Physical address = base + virtual address;
  - This allows multiple processes to be in memory with protection.
- Translation provides protection
  - Processes cannot talk about other processes' addresses, nor about the OS addresses
  - OS uses physical addresses directly
  - No translations

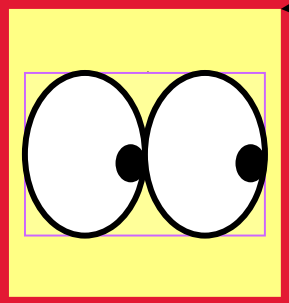
Dual-Mode Operation Revisited

- Translation tables offer protection if they cannot be altered by applications
- An application can only touch its address space under the user mode
- Hardware requires the CPU to be in the kernel mode to modify the address translation tables

Switching from Kernel to User Mode

- To run a user program, the kernel:
  - Creates a process and initialize the address space
  - Loads the program into the memory
  - Initializes translation tables
  - Sets the hardware pointer to the translation table
  - Sets the CPU to user mode
  - Jumps to the entry point of the program

To Run a Program



User level

Kernel level

Translation table

Hardware pointer

user mode

PC

Kernel Mode

- Voluntary
  - ***System calls***: a user process asks the OS to do something on the process's behalf
- Involuntary
  - Hardware interrupts (e.g., I/O)
  - Program exceptions (e.g., segmentation fault)

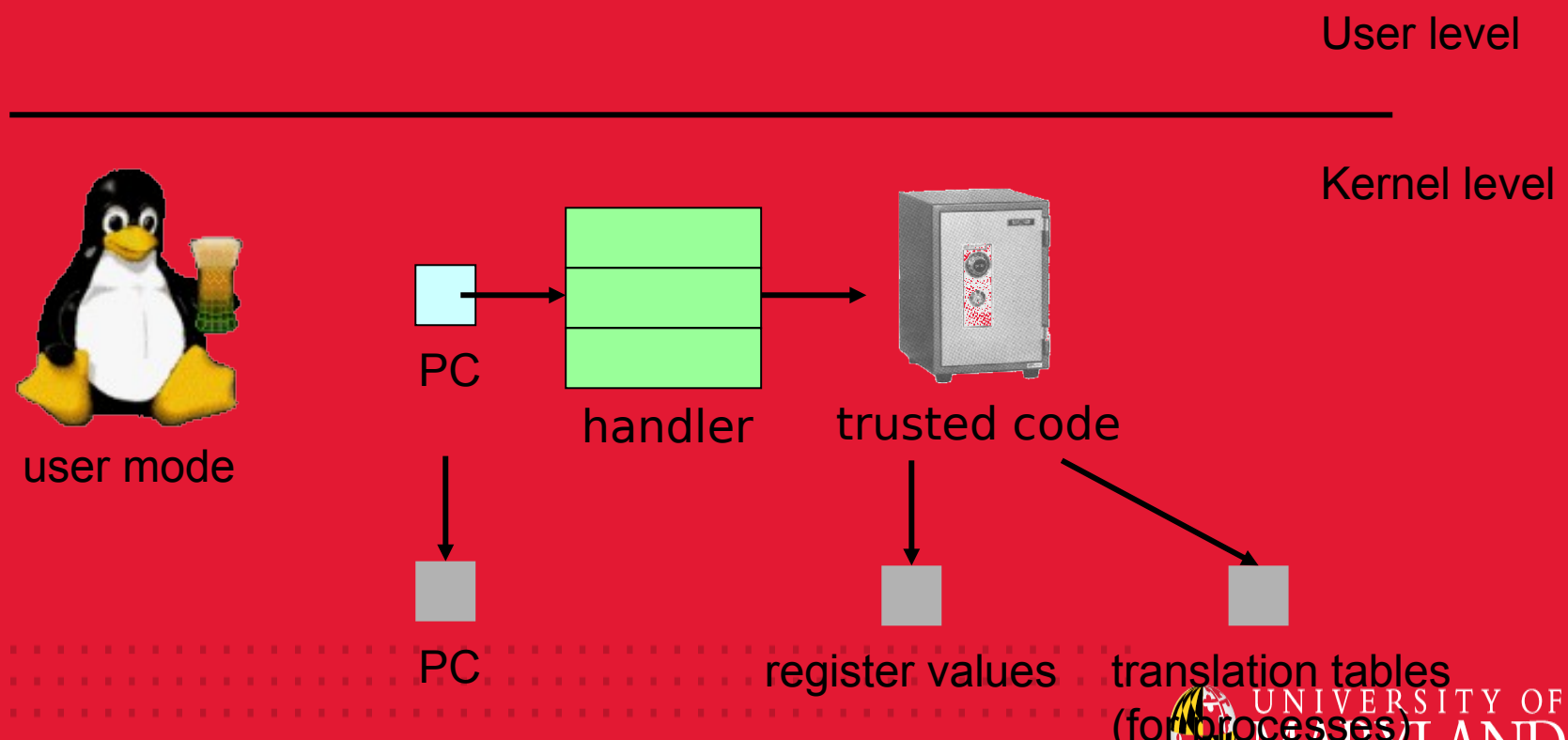
Kernel Mode

- For all cases, hardware atomically performs the following steps
  - Sets the CPU to kernel mode
  - Saves the current program counter
  - Jumps to the handler in the kernel
    - The handler saves old register values

Kernel Mode

- Context switching between processes
  - Need to save and restore pointers to translation tables
- To resume process execution
  - Kernel reloads old register values
  - Sets CPU to user mode
  - Jumps to the old program counter

User ☾ Kernel


User level
Kernel level
user mode
PC
handler
trusted code
PC
register values
translation tables
(for processes)

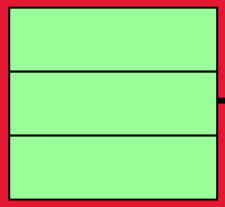
Kernel ☾ User

User level

Kernel level

set kernel mode

PC



handler

trusted code

PC

register values

translation tables
(for processes)

Secure Memory Management Rules

Rules for Memory Management

- Do not access freed memory
- Free dynamically allocated memory when not needed
- Allocate sufficient memory for an object
- Clear memory before freeing
- Make sure that sensitive information is not saved to disk

COUNTERMEASURES AND MITIGATIONS

Probabilistic Countermeasures

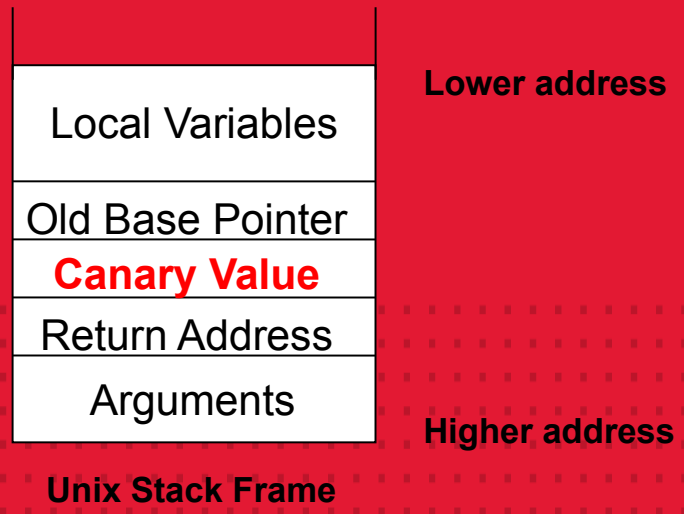
- Based on randomness
- Canary-based approach
  - Place a random number in memory
  - If the random number has been modified it will indicate an overflow
  - Check random number before performing action
  - If random number changed an overflow has occurred
- Obfuscation of memory addresses
- Address Space Layout Randomization
- Instruction Set Randomization

Canary-based approaches

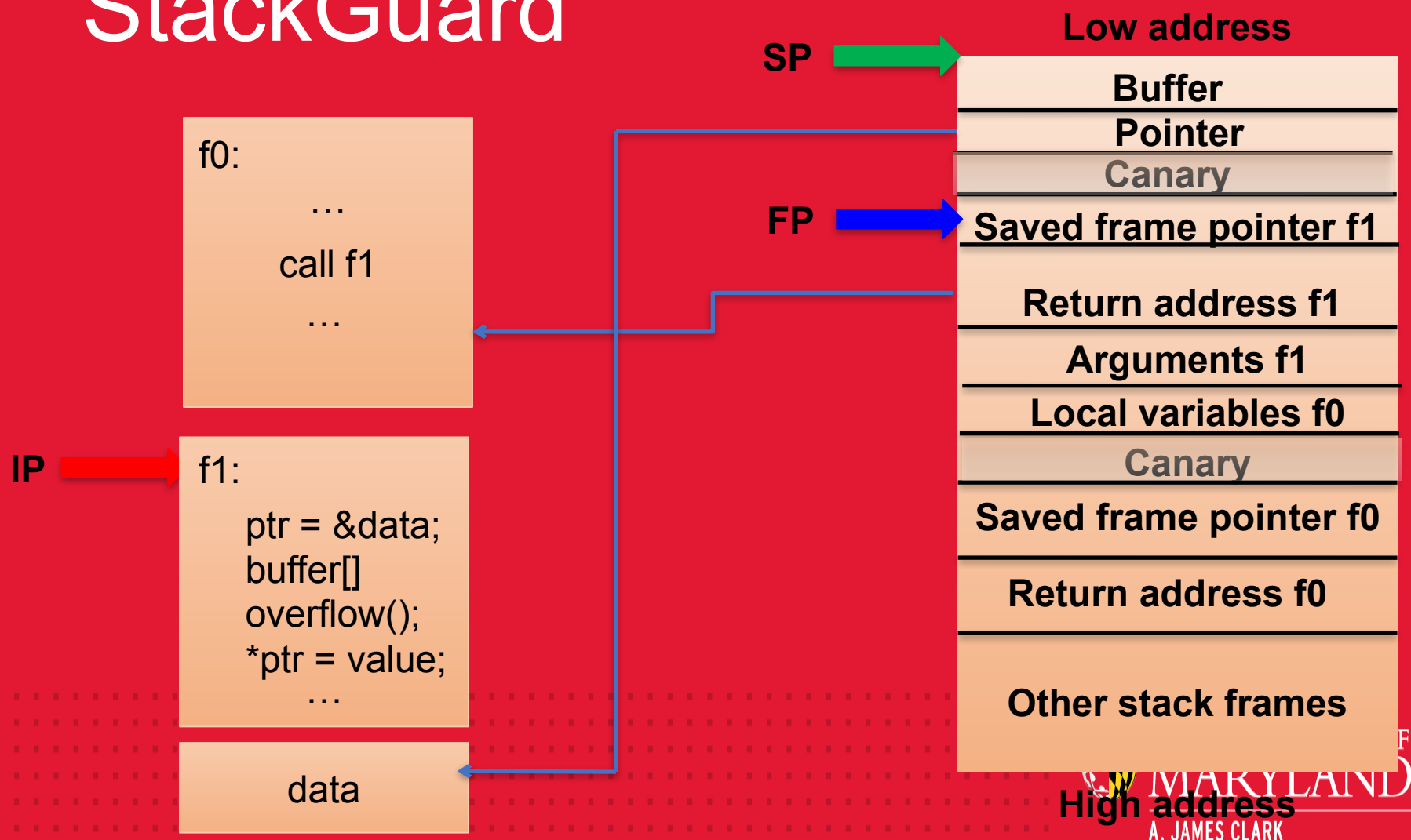
- StackGuard (SG): Crispin Cowan
  - Places random number before the return address when entering a function
  - Verified that the random number is unchanged when returning from the function
  - If it has changed, an overflow is assumed to occur and the program is terminated.

Preventing Stack Buffer Overflow

- Stack Guard
  - In a stack , a canary word is placed after return address whenever a function is called;
  - The canary will be checked before the function returns. If value of canary is changed , then it indicates an malicious behavior.



StackGuard


Low address
SP
Buffer
Pointer
Canary
FP
Saved frame pointer f1
Return address f1
Arguments f1
Local variables f0
Canary
Saved frame pointer f0
Return address f0
Other stack frames
High address
f0:
…
call f1
…
IP
f1:
ptr = &data;
buffer[]
overflow();
*ptr = value;
…
data

StackGuard

```
f0:
    …
    call f1
    …
```

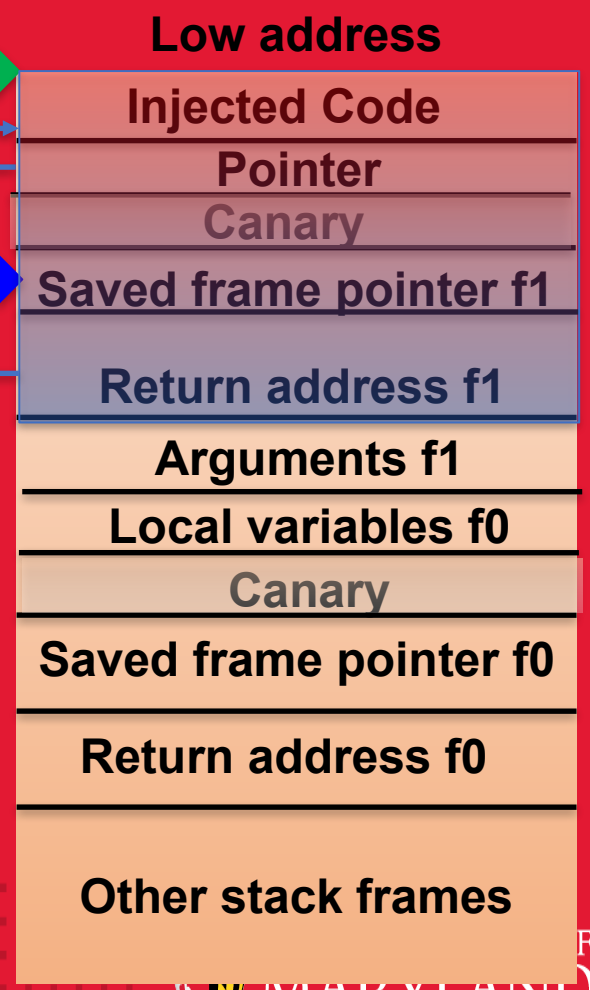
IP

```
f1:
    ptr = &data;
    buffer[]
    overflow();
    *ptr = value;
    …
```

data

SP

FP



StackGuard continued

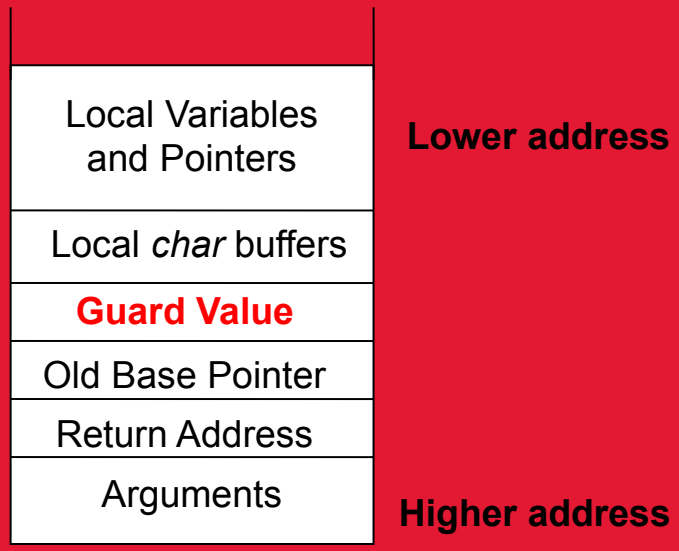
- Canary can still be intact if the attacker overwrites it with the correct value
  - Solution – use "*random canary*" value
  - Use "*terminator canary*" – consists of all string terminator sequences – NULL, '\r', '\n', -1…
- Attacker can still point to the 'return address' and change it, without worrying about the canary
  - This is a short-coming of StackGuard
  - Can be dealt with by XORing the canary value and 'return address' to detect if 'return address' has changed

Stack Shield

- Stack Shield
  - Copy RET address into an unoverflowable memory region;
  - The values of two RET addresses will be compared before a function returns;
  - If the values are different, then an malicious exploitation occurs;
  - Needs another stack-like data structure to maintain RET addresses.

Preventing stack buffer overflow (Cont.)

- ProPolice
  - Perhaps most sophisticated compiler protection
  - Same principle as StackGuard
  - Protects against indirect pointer overwriting by reorganizing the stack frame
    - All arrays are stored before all other data on the stack (i.e. right next to the random value)
    - Overflows will cause arrays to overwrite other arrays of the random value

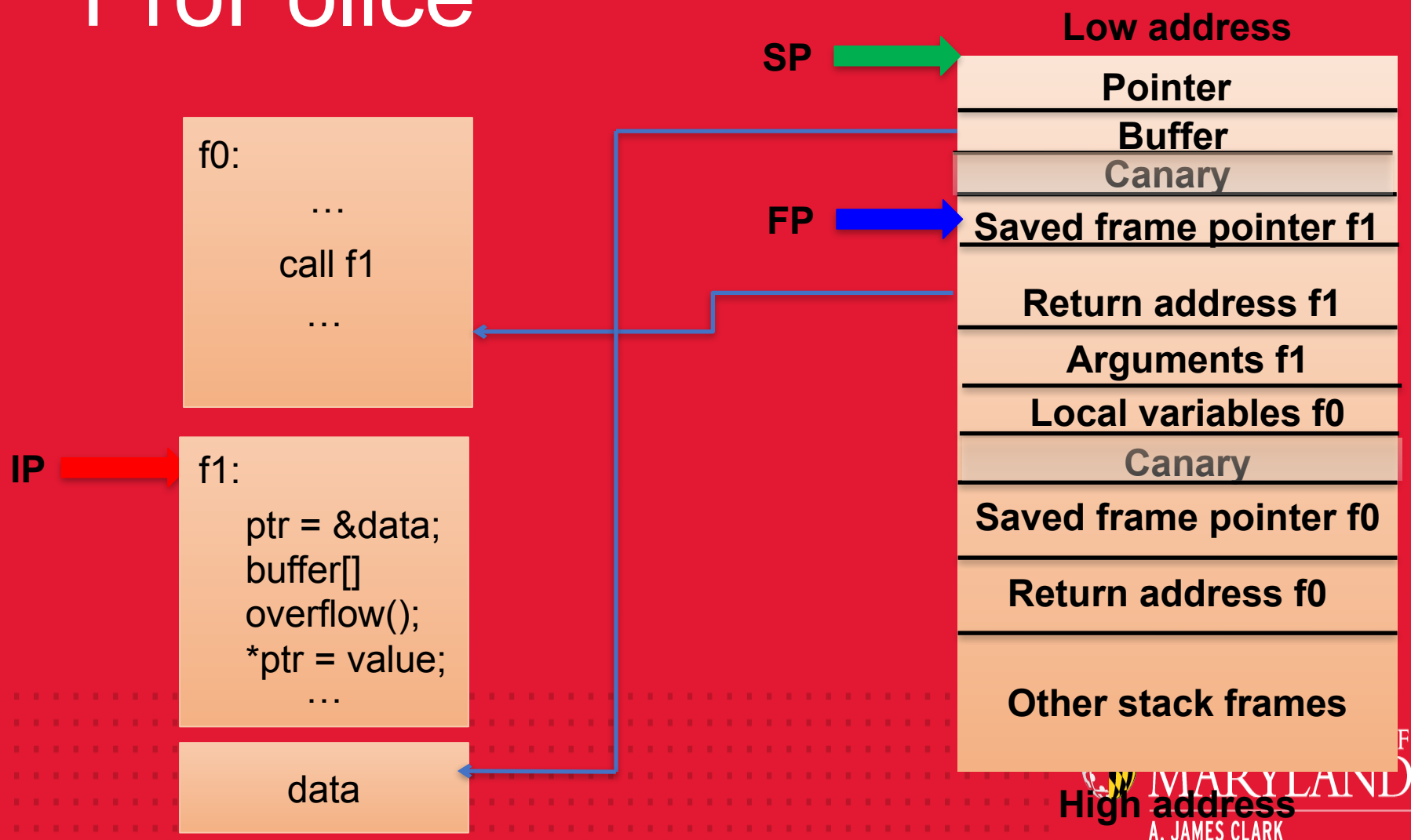


Unix Stack Frame

Propolice (continued)

- Rearrange local variables such that *char* buffers always are allocated at bottom addresses (top of the stack), and are guarded by a Guard Value
- Does not work fine with small buffers – somewhat unstable
- Part of GCC >= 4.1
- Stack Cookies in Visual Studio

ProPolice


Low address
SP
Pointer
Buffer
Canary
FP
Saved frame pointer f1
Return address f1
Arguments f1
Local variables f0
Canary
Saved frame pointer f0
Return address f0
Other stack frames
High address
f0:
…
call f1
…
IP
f1:
ptr = &data;
buffer[]
overflow();
*ptr = value;
…
data

Stackguard

```
f0:
    …
    call f1
    …
```

IP

```
f1:
    ptr = &data;
    buffer[]
    overflow();
    *ptr = value;
    …
```

data

SP

FP

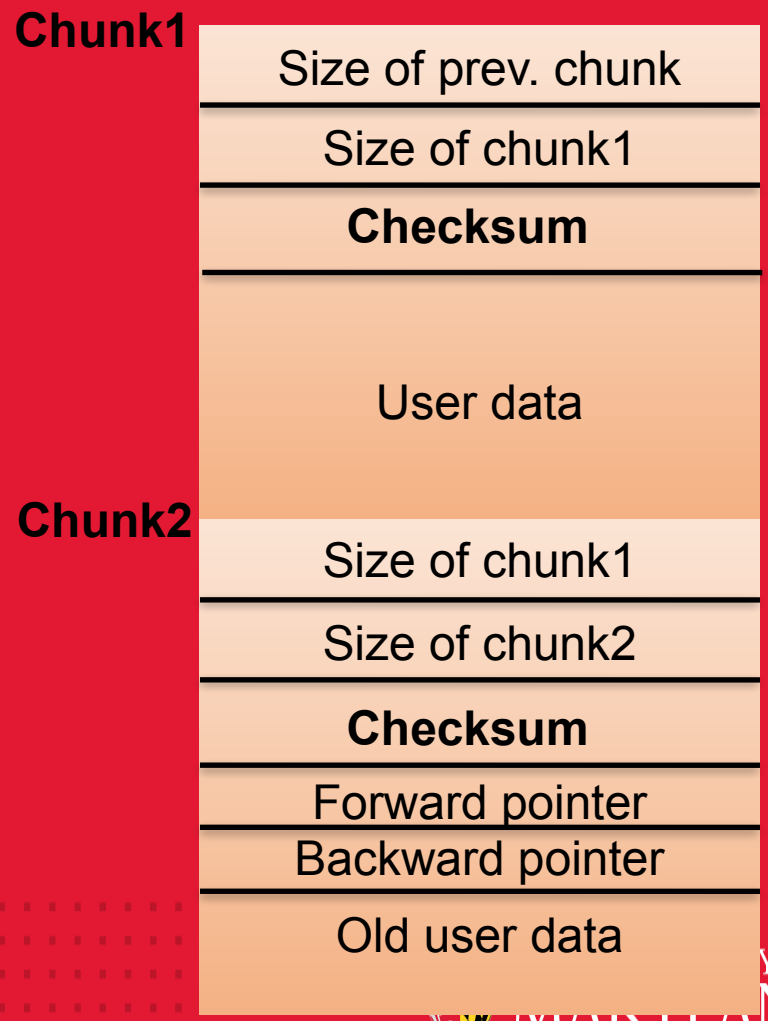
Low address

Pointer
Injected Code
Canary
Saved frame pointer f1
Return address f1
Arguments f1
Local variables f0
Canary
Saved frame pointer f0
Return address f0
Other stack frames

High address

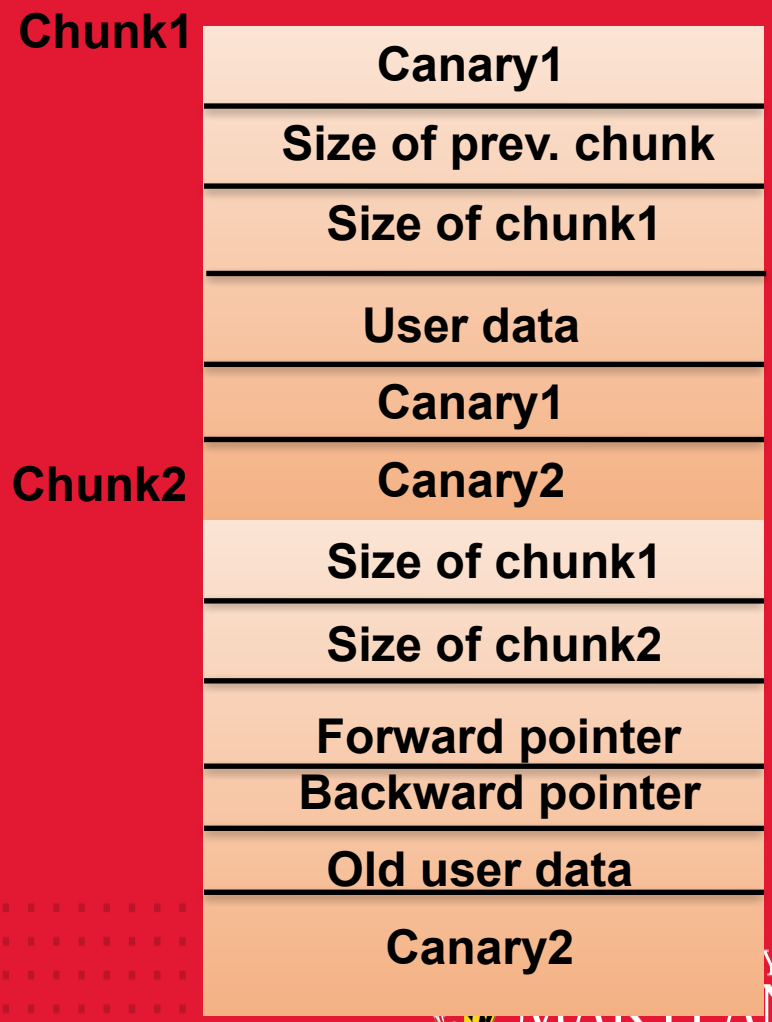
Heap Protector

- Adds checksum to the chunk information
- Checksum is XORed with a global random value
- On allocation the checksum is added
- On free (or other operations) checksum is calculated, XORed and compared



ContraPolice

- Stores a random value before and after the chunck
- Before exiting from a string copy operation, the random value is compared to the random value after
- If they are not the same an overflow has occurred



Problem with the Canary-Based Approach

- Random value can leak
- For SG: Indirect Pointer Overwriting
- For PP: overflow from one array to the other (e.g. array of char overwrites an array of pointers)
- For HP, SG, PP: 1 global random value
- CP: different random numbers per chunk
  - Only does string copy functions
- CP: no protection against overflow in loops

Differences between ProPolice and StackGuard

- Propolice and StackGuard have approached the same problem from opposite directions
- Canary
  - Places the canary word as a *buffer overflow* detector *only* at the top of auto variable regions containing “buffers”
  - StackGuard places the canary word as a stack smash detector at the bottom of every control region
- Randomness of canaries
  - Propolice uses random canaries
  - StackGuard uses terminator canaries

Obfuscation of Memory Addresses

- Based on random numbers
- Numbers used to ‘encrypt’ memory locations
- Usually XOR
  - a XOR b = c
  - c XOR b = a

Pointguard

- Protects all pointers by encrypting them (XOR) with a random value
- Decryption key is stored in a register
- Pointer is decrypted when loaded into a register
- Pointer is encrypted when loaded into memory
- Forces the compiler to do all memory access via registers
- Can be bypassed if the key or a pointer leaks
- Randomness can be lowered using a partial overwrite

Address Space Layout Landomization (ASLR)

- PaX – PageEXec Address Space Layout Randomization
  - Implemented in
    - Dragonfly BSD
    - Hardened Gentoo
    - Hardened Debian
- Compiler must generate PIC
- Randomizes the base address of the stack, head, code and shared memory segments
- Makes it harder for an attacker to know where in memory his code is located
- Can be bypassed if attackers can print out memory addresses – possible to derive the base address
- Implemented in Windows Vista/7+ & Linux >= 2.6.12

Bypassing ASLR with heap spraying

- If attacker can control memory allocation in the program (e.g. browser via javascript)
- Allocate a significant amount of memory
  - Ex: 1GB or 2GB
  - Fill memory with a bunch of NOPs, place shell code at the end
  - Reduces amount of randomization offered by ASLR
  - Jumping anywhere in the NOPs will cause the shell code to be executed eventually

Separation and replication of information

- Replicate valuable control-flow information
  - Copy control-flow information to other memory
  - Copy back or compare before using
- Separate control-flow information from other data
  - Write control-flow information to other places in memory
  - Prevents overflows from overwriting control flow information
- These approaches do not rely on randomness

Separation of Information

- DNmalloc
  - Does not rely on random numbers
  - Protection is added by separating the chunk information from the chunk
  - Chunk information is stored in separate regions protected by guard pages
  - Chunk is linked to its information through a hash table
  - Fast: performance impact vs. dlmalloc: -10% to +5%
  - Used as the default allocator for Samhain (open source file integrity IDS)

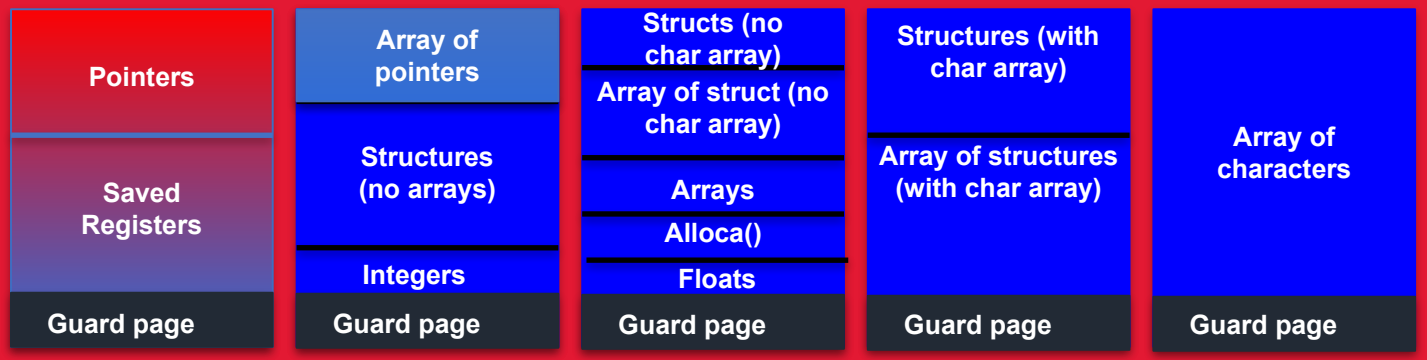
DNmalloc

Low addresses
Heap Data
Heap Data
Heap Data
Heap Data
Heap Data
Heap Data
Heap Data
Heap Data
Heap Data
Heap Data
High addresses
Hashtable
Guard Page
Ptr to chunkinfo
Ptr to chunkinfo
Ptr to chunkinfo
Ptr to chunkinfo
Ptr to chunkinfo
Ptr to chunkinfo
Ptr to chunkinfo
Chunkinfo region
Guard Page
Management Info
Management Info
Management Info
Management Info
Control data
Regular data

Multistack

- Does not rely on random numbers
- Separates the stack into multiple stacks, 2 criteria:
  - Risk of data being an attack target (target value)
  - Risk of data being used as an attack vector (source value)
    - Return address: target → high; source → Low
    - Arrays of character: target → Low; source → High
  - Default: 5 stacks, separated by guard pages
    - Stacks can be reduced by using selective bounds checking
    - To reduce source risk: ideally use 2 stacks
  - Fast – maximum performance overhead: 2 – 3% (usually 0)

DNstack



- Stacks are at a fixed location from each other
- If source risk can be reduced: maybe only 2 stacks
  - Map stack 1,2 onto stack one
  - Map stack 3,4,5 onto stack two

Non-Executable memory

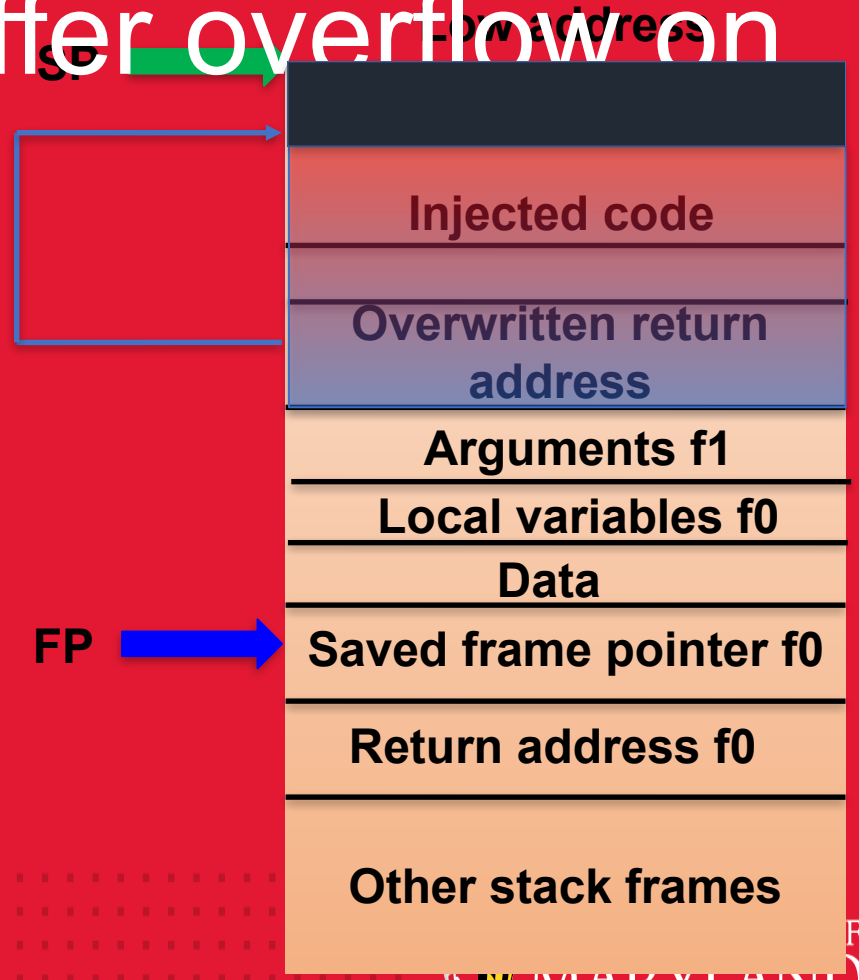
- Pages of memory can be marked executable, writeable, and readable
- Older Intel processors would not support the executable bit which meant that readable meant executable
- Eventually the bit was implemented, allowing the OS to mark data pages (such as the stack and heap writeable but not executable)
- OpenBSD takes it further with the implementation of W^X (writeable XOR executable)
- Programs doing JIT have memory that is both executable and writeable

Stack-based buffer overflow on NX

```
f0:
    …
    call f1
    …
```

IP

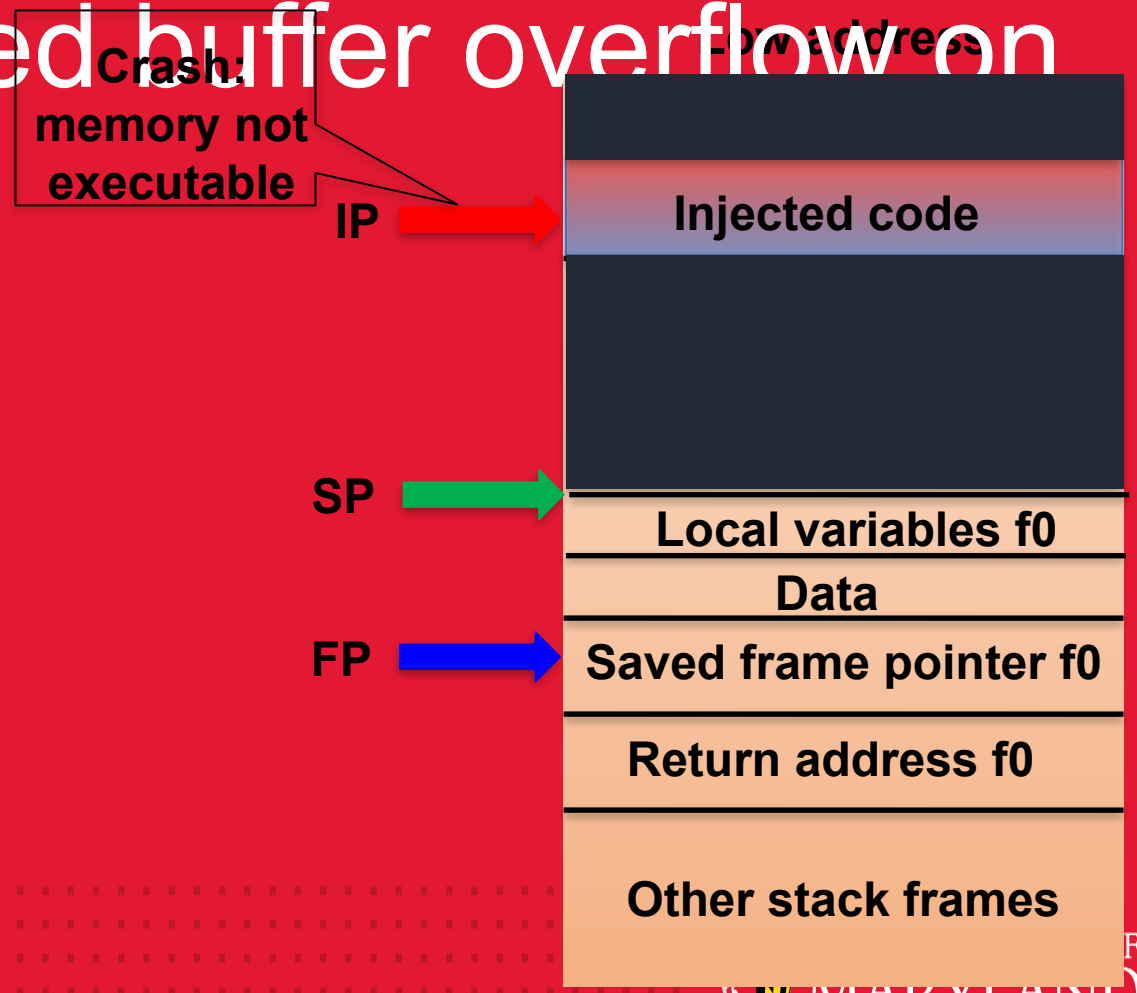
```
f1:
    buffer[]
    overflow();

    …
```



Stack-based buffer overflow on NX

```
f0:
    …
    call f1
    …
```

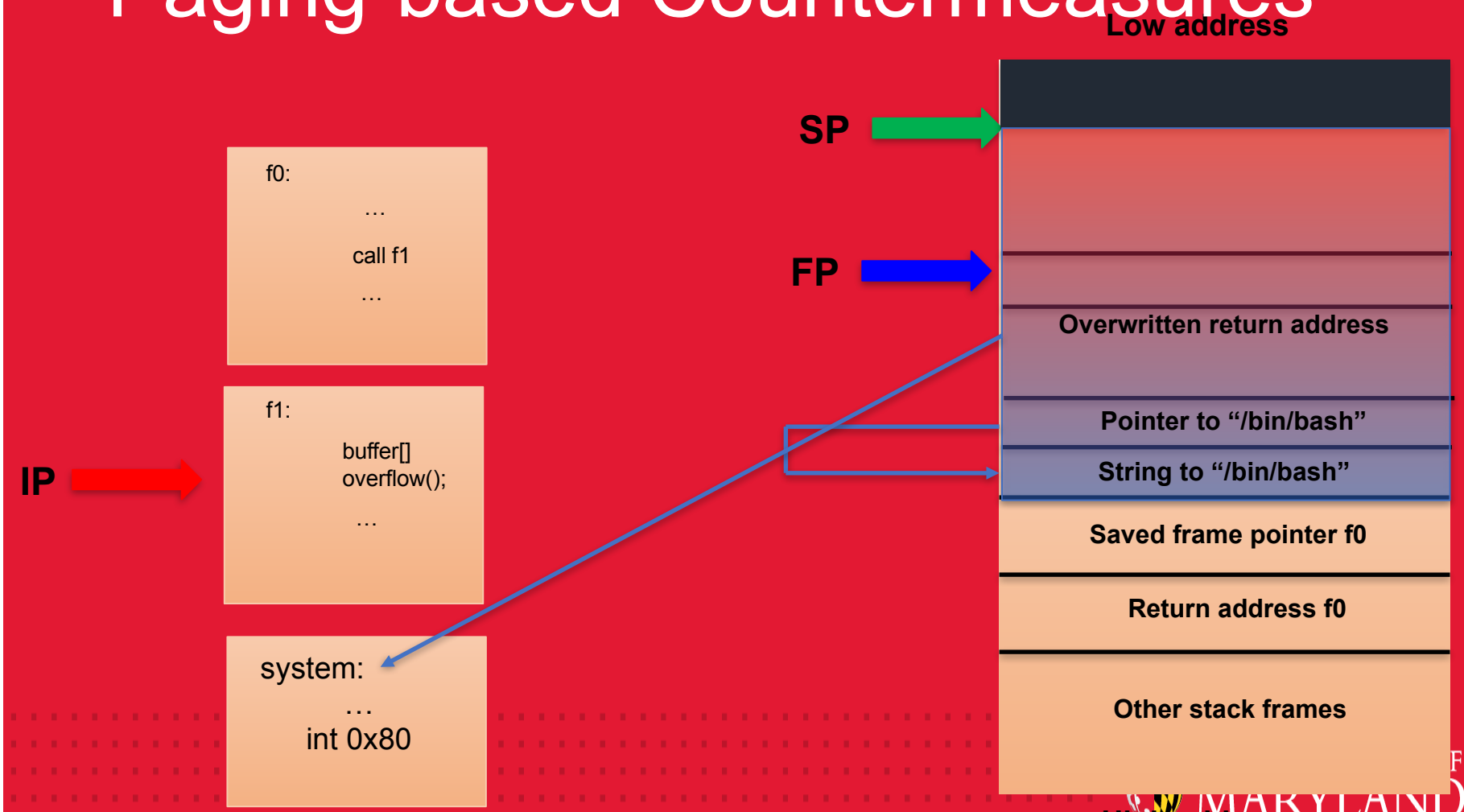
```
f1:
    buffer[]
    overflow();

    …
```



Bypassing Non-executable Memory

- Early exploits would return to existing functions (called return-to-libc) to bypass these countermeasures
  - Places the arguments on the stack and then places the address of the function as the return address
    - This simulates a function call
  - Ex: calling system ("/bin/bash") would place the address of the executable code for system() as return address and would place a pointer to the string /bin/bash on the stack

Paging-based Countermeasures



Secure Memory Management Functions

- Add library function to control security and performance of memory manager
- Possible properties to control:
  - Clear freed memory
  - Randomize address of pointers returned by allocation functions
  - Disable caching
  - If the argument does not match a pointer earlier returned by a memory management function, or if the space has been deallocated by a call to **free()** or **realloc()**, no action occurs.

Upcoming

- Homework 3 available on Thursday October 26th